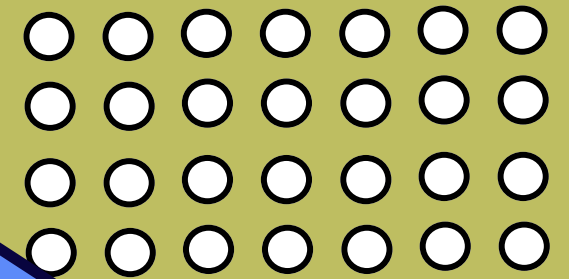
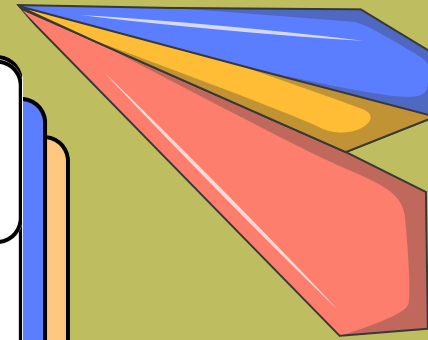
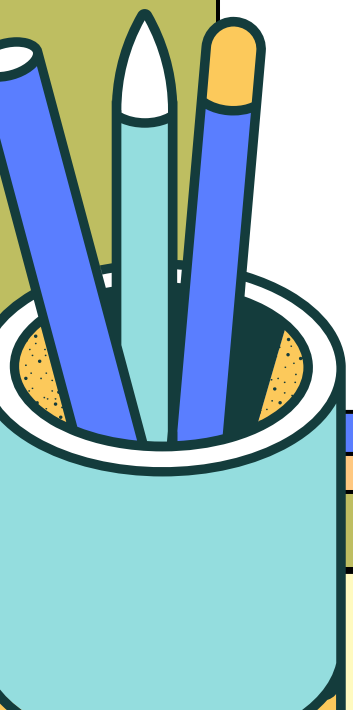


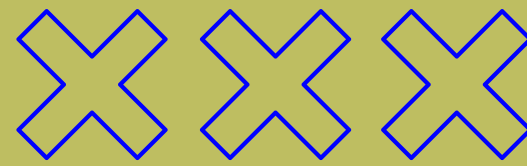
PEMROGRAMAN BERORIENTASI OBJEK

Sistem Manajemen Perpustakaan Digital

NYAWIT GROUP



Anggota kelompok



Zulfan Hanif Ihsani - (103112430221)



Daffa Tsaqifna Fauztsany - (103112430032)



Bayu Wandana - (103112430159)



Andika Arya Saputra - (103112400247)



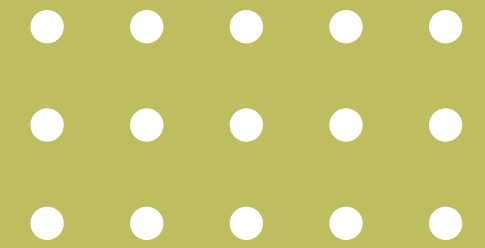
Tio Armani - (103112430225)



Chadafya Putra zulfikar - (103112430173)



Sistem Perpustakaan Digital Berbasis



Sistem Perpustakaan Digital adalah aplikasi berbasis web yang dibuat untuk membantu pengelolaan perpustakaan agar menjadi lebih mudah, cepat, dan teratur.

Melalui aplikasi ini, admin dapat mengelola data buku, seperti menambahkan buku baru, melihat daftar buku, mengubah informasi buku, dan menghapus buku yang sudah tidak digunakan. Selain itu, admin juga dapat mengatur kategori buku dan memantau data yang ada di perpustakaan.

Di sisi lain, anggota dapat melihat daftar buku yang tersedia, mengetahui jumlah stok buku, dan melakukan peminjaman buku secara online tanpa harus melakukan pencatatan secara manual.

Seluruh data disimpan di dalam database sehingga informasi menjadi lebih aman, rapi, dan mudah dikelola.

Manfaat Aplikasi

- Mempermudah admin dalam mengelola data buku.
- Membantu anggota mencari dan meminjam buku dengan lebih mudah.
- Mengurangi penggunaan pencatatan secara manual.
- Menyimpan data dengan lebih aman di dalam database.
- Mempercepat proses pengelolaan perpustakaan.
- Memudahkan melihat stok buku yang masih tersedia.
- Membuat pengelolaan perpustakaan menjadi lebih rapi dan efisien.

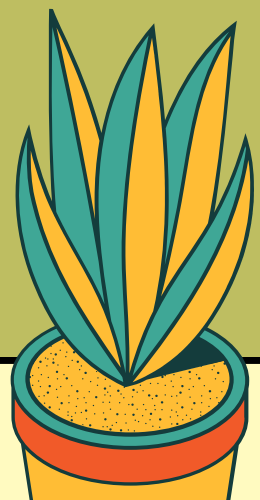
Book Controller

BookController.java



BookController berfungsi sebagai penghubung antara pengguna dan sistem. Semua permintaan yang berkaitan dengan buku, seperti melihat, menambah, mengubah, dan menghapus buku, pertama kali diterima oleh Controller sebelum diproses lebih lanjut oleh Service.

```
1 package com.tup.temp.controller;
2
3 import java.util.List;
4
5 import org.springframework.web.bind.annotation.*;
6
7 import com.tup.temp.entity.Book;
8 import com.tup.temp.service.BookService;
9
10 @RestController
11 @RequestMapping("/book")
12 public class BookController {
13
14     private final BookService bookService;
15
16     public BookController(BookService bookService) {
17         this.bookService = bookService;
18     }
19
20     @PostMapping("/add")
21     public Book addBook(@RequestBody Book book) {
22         return bookService.save(book);
23     }
24
25     @GetMapping("/all")
26     public List<Book> getAllBooks() {
27         return bookService.getAll();
28     }
29     @GetMapping("/{id}")
30     public Book getBook(@PathVariable Long id) {
31         return bookService.getById(id);
32     }
33
34     @PutMapping("/update/{id}")
35     public Book updateBook(
36         @PathVariable Long id,
37         @RequestBody Book updatedBook) {
38
39         Book book = bookService.getById(id);
40
41         book.setTitle(updatedBook.getTitle());
42         book.setAuthor(updatedBook.getAuthor());
43         book.setStock(updatedBook.getStock());
44         book.setCategory(updatedBook.getCategory());
45
46         return bookService.save(book);
47     }
48     @DeleteMapping("/delete/{id}")
49     public void deleteBook(@PathVariable Long id) {
50         bookService.delete(id);
51     }
52 }
```



BookService

BookService.java

BookService merupakan bagian yang menjalankan proses utama pengelolaan buku. Service bertugas mengolah data yang diterima dari Controller sebelum data tersebut disimpan atau diambil dari database."

```
1 package com.tup.temp.service;
2
3 import java.util.List;
4
5 import org.springframework.stereotype.Service;
6
7 import com.tup.temp.entity.Book;
8 import com.tup.temp.repository.BookRepository;
9
10 @Service
11 public class BookService {
12
13     private final BookRepository bookRepository;
14
15     public BookService(BookRepository bookRepository) {
16         this.bookRepository = bookRepository;
17     }
18
19     public Book save(Book book) {
20         return bookRepository.save(book);
21     }
22
23     public List<Book> getAll() {
24         return bookRepository.findAll();
25     }
26
27     public Book getById(Long id) {
28         return bookRepository.findById(id).orElseThrow();
29     }
30
31     public void delete(Long id) {
32         bookRepository.deleteById(id);
33     }
34
35 }
```




Book



Book.java

Book merupakan Entity yang digunakan untuk menyimpan informasi buku. Setiap objek Book mewakili satu data buku yang nantinya akan disimpan pada tabel database

```
1 public String getTitle() {
2     return title;
3 }
4
5 public void setTitle(String title) {
6     this.title = title;
7 }
8
9 public String getAuthor() {
10    return author;
11 }
12
13 public void setAuthor(String author) {
14    this.author = author;
15 }
16
17 public int getStock() {
18    return stock;
19 }
20
21 public void setStock(int stock) {
22    this.stock = stock;
23 }
24
25 public Category getCategory() {
26    return category;
27 }
28
29 public void setCategory(Category category) {
30    this.category = category;
31 }
32 }
```

```
package com.tup.temp.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "books")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    private String author;

    private int stock;

    @ManyToOne
    @JoinColumn(name = "category_id")
    private Category category;

    public Book() {
    }

    public Book(String title, String author, int stock, Category category) {
        this.title = title;
        this.author = author;
        this.stock = stock;
        this.category = category;
    }

    public Long getId() {
        return id;
    }
}
```



Book Repository



BookRepository.java

BookRepository berfungsi sebagai penghubung antara aplikasi dan database. Semua proses penyimpanan, pengambilan, pengubahan, dan penghapusan data buku dilakukan melalui Repository.

```
1 package com.tup.temp.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 import com.tup.temp.entity.Book;
6
7 public interface BookRepository extends JpaRepository<Book, Long> {
8
9 }
```



books



books.html

Halaman books.html pada admin digunakan untuk menampilkan seluruh data buku yang tersedia di perpustakaan. Melalui halaman ini admin dapat mengelola data buku secara lengkap.

Books Management

Add Book

Title	Author	Stock	Category	Actions
wgtik	pak maulidi	30	Programming	Edit Delete
pemograman java	pak dany	200	Programming	Edit Delete
The Innovators	Walter Isaacson	25	Technology	Edit Delete
Algoritma dan Pemrograman	Rinaldi Munir	15	Programming	Edit Delete
Otodidak Pemrograman Python	Jubilee Enterprise	25	Programming	Edit Delete
Laut Bercerita	Leila S. Chudori	40	Novel	Edit Delete
Gadis Kretek	Ratih Kumala	40	Novel	Edit Delete
Cantik Itu Luka	Eka Kurniawan	15	Novel	Edit Delete
Ronggeng Dukuh Paruk	Ahmad Tohari	20	Novel	Edit Delete
Catatan Seorang Demonstan	Soe Hok Gie	28	History	Edit Delete
Kronik Revolusi Indonesia	Pramoedya Ananta Toer	8	History	Edit Delete
Mestakung: Semesta Mendukung	Yohanes Surya	12	Science	Edit Delete
Asal-Usul Manusia Indonesia	Harry Widiyanto	11	Science	Edit Delete
Kecerdasan Buatan (Artificial Intelligence)	Suyanto	22	Technology	Edit Delete
Cloud Computing: Teori dan Implementasi	Iwan Sofana	16	Technology	Edit Delete
Disruption: Menghadapi Perubahan Zaman	Rhenald Kasali	30	Technology	Edit Delete

Back



add-book



add-book.html

Halaman add-book digunakan untuk memasukkan data buku baru ke dalam sistem. Setelah data diisi dan disimpan, sistem akan menyimpan data tersebut ke database sehingga buku dapat ditampilkan pada daftar buku."

Add Book

Title

Disruption: Menghadapi Perubahan Zaman

Author

Rhenald Kasali

Stock

20

Category

Programming

Programming

Novel

History

Science

Technology



edit-book



edit-book.html

Halaman edit-book digunakan untuk memperbarui informasi buku yang sudah tersimpan. Dengan fitur ini admin dapat memastikan data buku selalu sesuai dengan kondisi yang sebenarnya."

Edit Book Stock

Stock

Save

Cancel



books



books.html

Halaman books pada member digunakan untuk menampilkan koleksi buku yang tersedia. Melalui halaman ini anggota dapat melihat informasi buku dan melakukan proses peminjaman tanpa memiliki akses untuk mengubah data buku

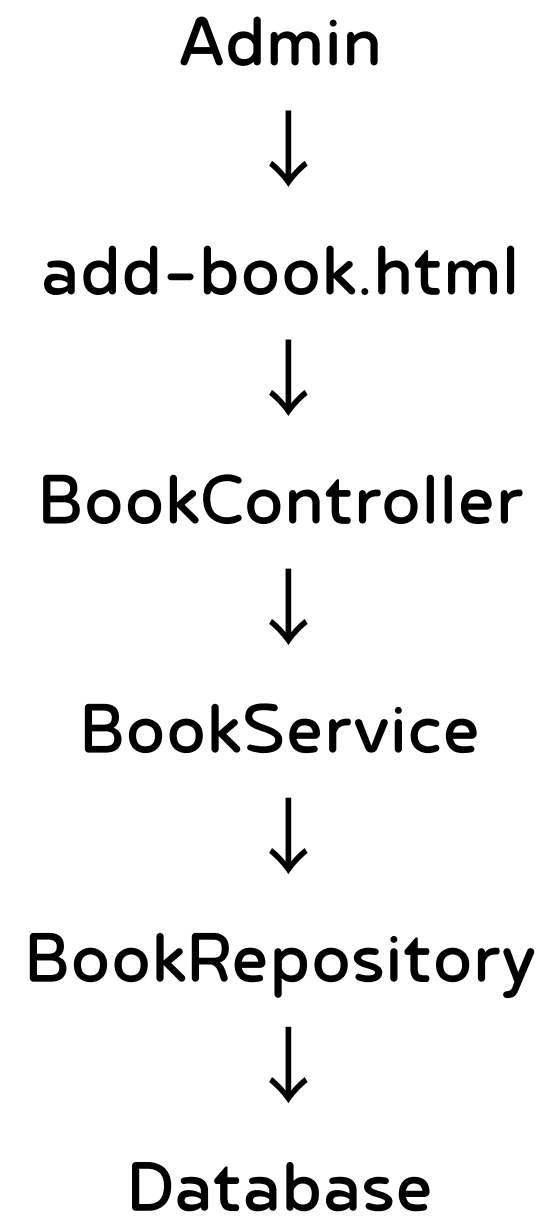
Browse Books

Back

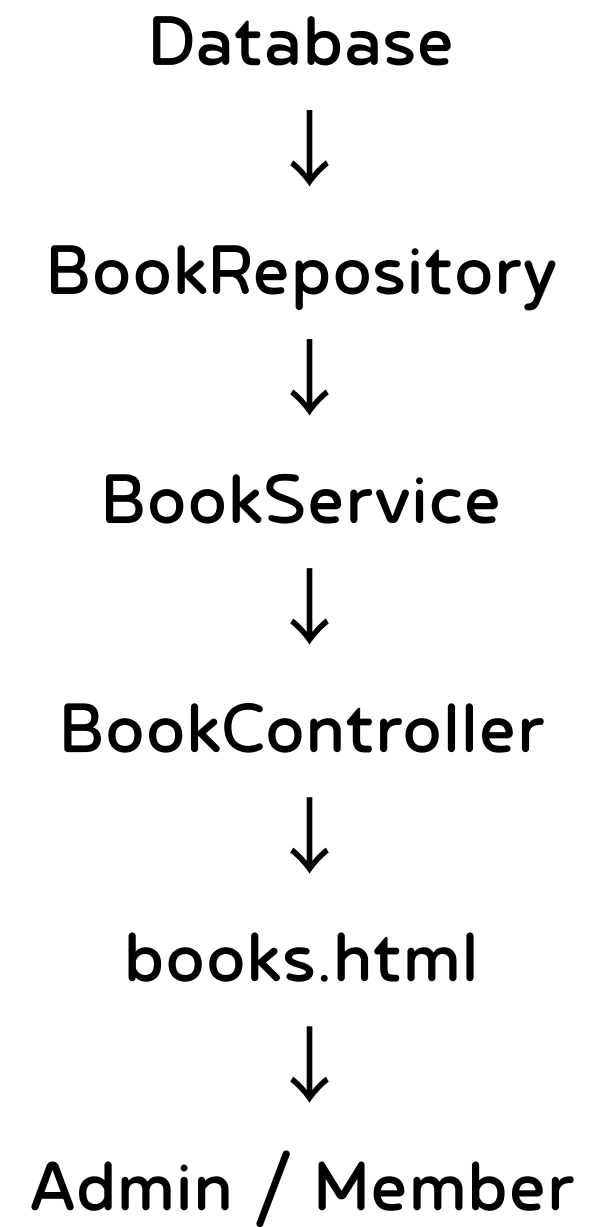
Title	Author	Stock	Category	Action
wgtik	pak maulidi	30	Programming	Borrow
pemograman java	pak dany	200	Programming	Borrow
The Innovators	Walter Isaacson	25	Technology	Borrow
Algoritma dan Pemrograman	Rinaldi Munir	15	Programming	Borrow
Otodidak Pemrograman Python	Jubilee Enterprise	25	Programming	Borrow
Laut Bercerita	Leila S. Chudori	40	Novel	Borrow
Gadis Kretek	Ratih Kumala	40	Novel	Borrow
Cantik Itu Luka	Eka Kurniawan	15	Novel	Borrow
Ronggeng Dukuh Paruk	Ahmad Tohari	20	Novel	Borrow
Catatan Seorang Demonstan	Soe Hok Gie	28	History	Borrow
Kronik Revolusi Indonesia	Pramoedya Ananta Toer	8	History	Borrow
Mestakung: Semesta Mendukung	Yohanes Surya	12	Science	Borrow
Asal-Usul Manusia Indonesia	Harry Widiyanto	11	Science	Borrow
Kecerdasan Buatan (Artificial Intelligence)	Suyanto	22	Technology	Borrow
Cloud Computing: Teori dan Implementasi	Iwan Sofana	16	Technology	Borrow
Disruption: Menghadapi Perubahan Zaman	Rhenald Kasali	30	Technology	Borrow



CREATE (Menambah Buku)

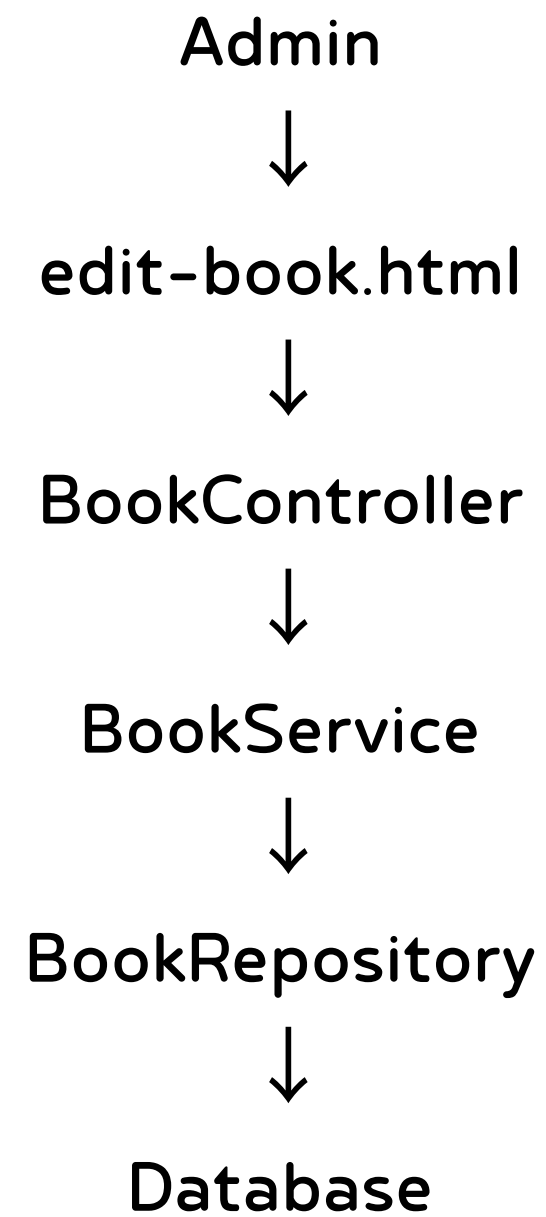


READ (Menampilkan Data Buku)

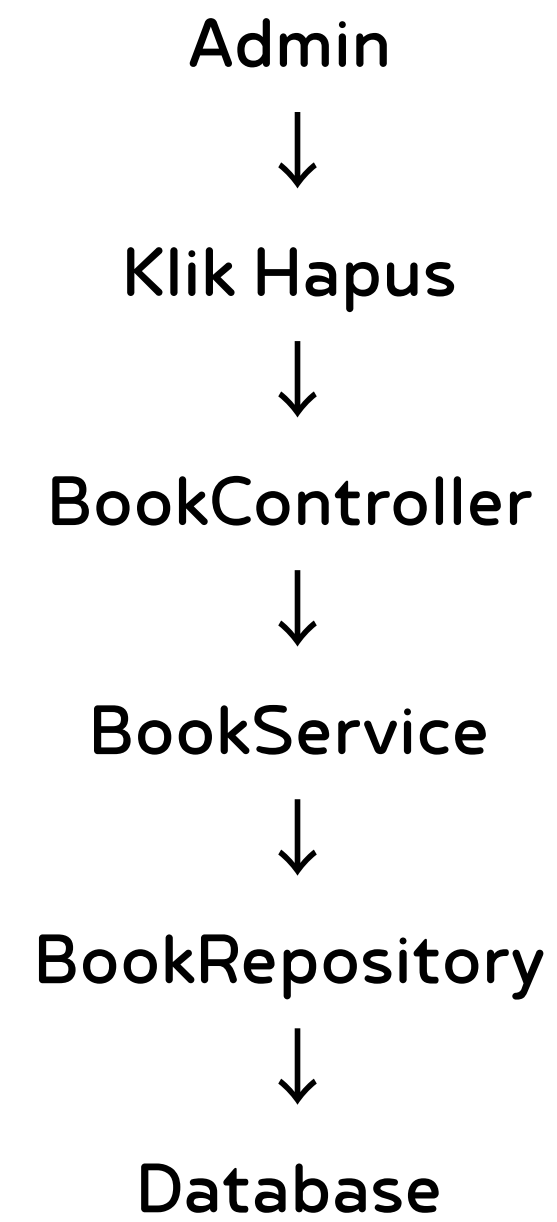




UPDATE (Mengubah Data Buku)



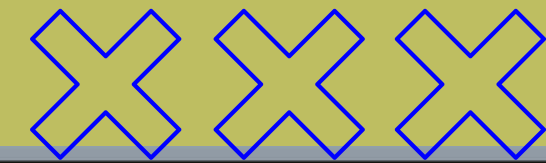
DELETE (Menghapus Data Buku)



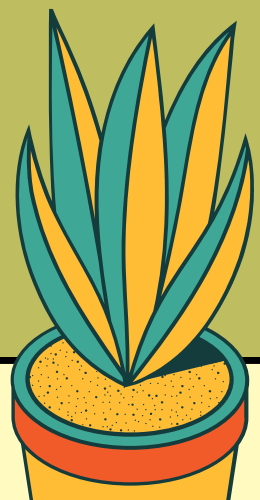
Booking Entity



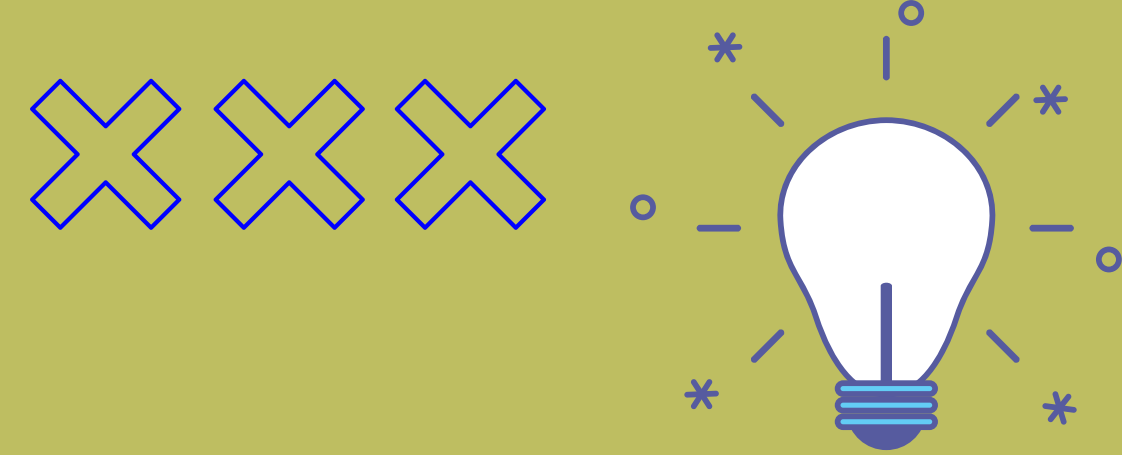
Entity Booking digunakan untuk menyimpan data transaksi peminjaman dan pengembalian buku. Data yang disimpan meliputi pengguna yang meminjam, buku yang dipinjam, tanggal peminjaman, tanggal jatuh tempo, dan status pengembalian serta informasi keterlambatan (expired).



```
1 package com.tup.temp.entity;
2
3 import java.time.LocalDate;
4
5 import jakarta.persistence.*;
6
7 @Entity
8 @Table(name = "bookings")
9 public class Booking {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @ManyToOne
16     @JoinColumn(name = "user_id")
17     private User user;
18
19     @ManyToOne
20     @JoinColumn(name = "book_id")
21     private Book book;
22
23     private LocalDate borrowDate;
24
25     private LocalDate dueDate;
26
27     private LocalDate returnDate;
28
29     private boolean expired;
30
31     private String status;
32 }
```



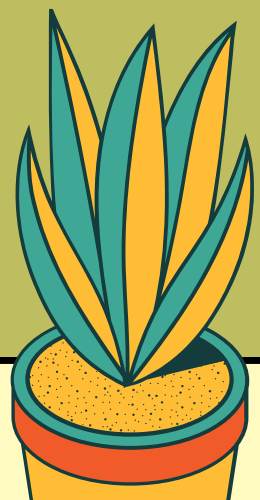
Booking Repository



Booking Repository berfungsi sebagai penghubung antara aplikasi dan database. Repository digunakan untuk menyimpan, mengambil, dan mencari data peminjaman berdasarkan pengguna yang sedang login.



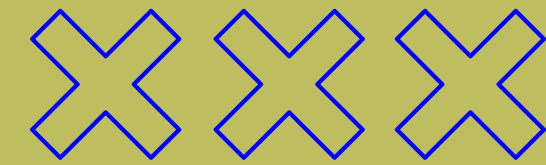
```
1 package com.tup.temp.repository;
2
3 import java.util.List;
4
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import com.tup.temp.entity.Booking;
8
9 public interface BookingRepository extends JpaRepository<Booking, Long> {
10     List<Booking> findByUserId(Long userId);
11 }
```



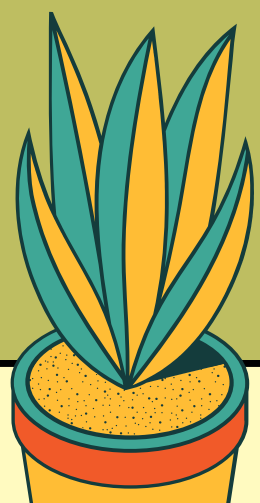
Booking Service



Booking Service merupakan lapisan logika bisnis yang menangani proses penyimpanan, pengambilan data, serta pencarian data peminjaman berdasarkan pengguna tertentu sebelum diteruskan ke controller.



```
1 package com.tup.temp.service;
2
3 import java.util.List;
4
5 import org.springframework.stereotype.Service;
6
7 import com.tup.temp.entity.Booking;
8 import com.tup.temp.repository.BookingRepository;
9
10 @Service
11 public class BookingService {
12
13     private final BookingRepository bookingRepository;
14
15     public BookingService(BookingRepository bookingRepository) {
16         this.bookingRepository = bookingRepository;
17     }
18
19     public Booking save(Booking booking) {
20         return bookingRepository.save(booking);
21     }
22
23     public List<Booking> getAll() {
24         return bookingRepository.findAll();
25     }
26
27     public Booking getById(Long id) {
28         return bookingRepository.findById(id).orElseThrow();
29     }
30     public List<Booking> getUserId(Long userId) {
31         return bookingRepository.findById(userId);
32     }
33 }
```





Proses Peminjaman Buku

Code : BookingController.java

```
1  if (book.getStock() <= 0) {  
2      throw new RuntimeException("Book is out of stock");  
3  }  
4  book.setStock(book.getStock() - 1);  
5  bookRepository.save(book);  
6  
7  Booking booking = new Booking();  
8  
9  booking.setUser(user);  
10 booking.setBook(book);  
11 booking.setBorrowDate(request.getBorrowDate());  
12 booking.setDueDate(request.getDueDate());  
13 booking.setExpired(false);  
14 booking.setStatus("BORROWED");  
15  
16 return bookingService.save(booking);  
17 }
```

Saat pengguna melakukan peminjaman, sistem akan memverifikasi data pengguna dan buku terlebih dahulu. Jika stok buku masih tersedia, stok akan dikurangi satu dan data peminjaman akan disimpan dengan status BORROWED.

My Borrowings

[Back](#)

Book	Borrow Date	Due Date	Status
SagaraS	2026-06-24	2026-07-01	BORROWED
SagaraS	2026-06-24	2026-07-01	BORROWED
Spring Boot Guide	2026-06-24	2026-07-01	BORROWED
Introduction to Algorithms	2026-06-24	2026-07-01	BORROWED



Proses Pengembalian Buku

Code : BookingController.java

Saat buku dikembalikan, sistem akan menambahkan kembali stok buku dan mengubah status peminjaman menjadi RETURNED sehingga buku dapat dipinjam kembali oleh pengguna lain.



```
1  @PostMapping("/return/{id}")
2      public Booking returnBook(@PathVariable Long id) {
3
4      Booking booking = bookingService.getById(id);
5
6      Book book = booking.getBook();
7
8      book.setStock(book.getStock() + 1);
9      bookRepository.save(book);
10
11     booking.setStatus("RETURNED");
```

My Borrowings

Back

Book	Borrow Date	Due Date	Status
SagaraS	2026-06-24	2026-06-22	RETURNED
SagaraS	2026-06-24	2026-07-01	RETURNED
Spring Boot Guide	2026-06-24	2026-07-01	BORROWED
Introduction to Algorithms	2026-06-24	2026-07-01	RETURNED

Keterlambatan dan Denda

Sistem melakukan pengecekan keterlambatan saat pengembalian buku. Jika tanggal pengembalian melewati tanggal jatuh tempo, sistem akan membuat data denda secara otomatis berdasarkan jumlah hari keterlambatan.

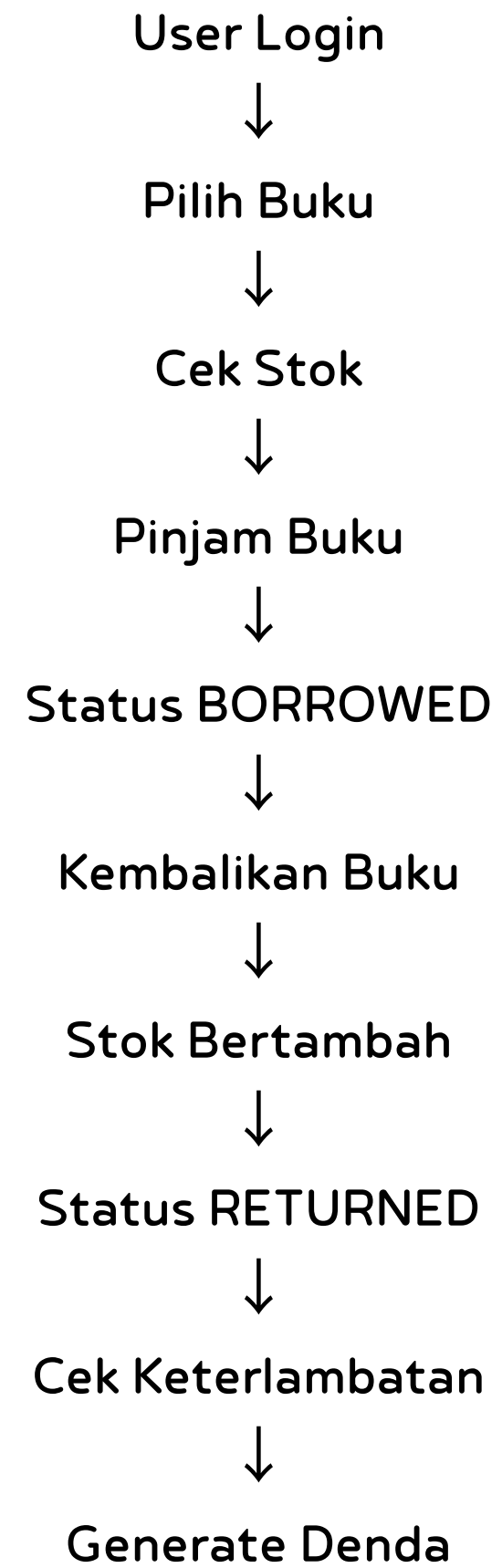
```
1  if (LocalDate.now().isAfter(booking.getDueDate())) {
2
3      Fine fine = new Fine();
4
5      fine.setBooking(booking);
6
7      long daysLate = ChronoUnit.DAYS.between(
8          booking.getDueDate(),
9          LocalDate.now()
10     );
11
12     fine.setAmount(daysLate * 1000);
13
14     fine.setPaid(false);
15
16     fineRepository.save(fine);
17 }
18
19 return bookingService.save(booking);
20 }
```

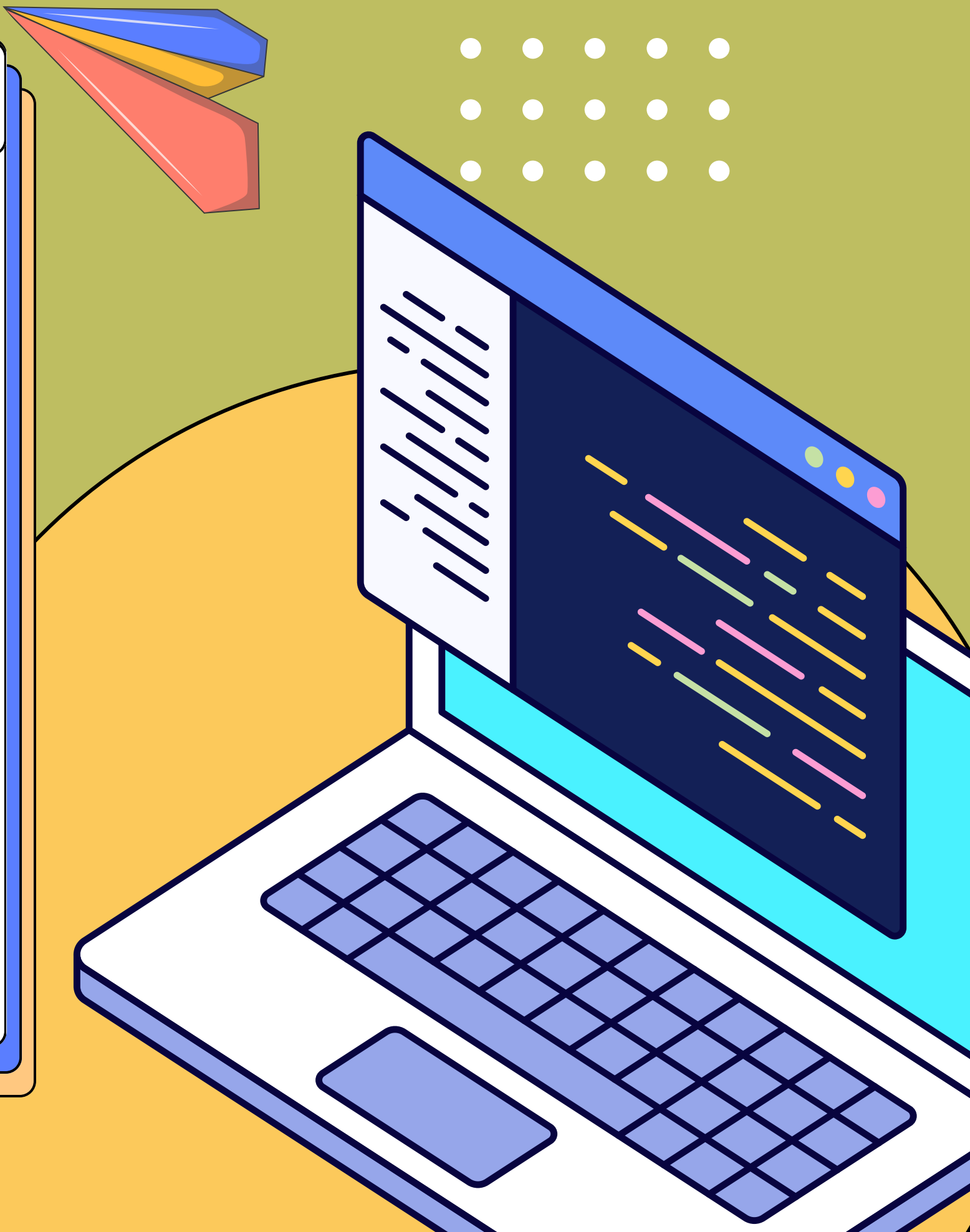
My Fines

[Back](#)

Booking ID	Amount	Paid
3	2000	false

Alur Sistem





**Terima
Kasih!**